PYTHON FOR ACTUARIES · NOTEBOOK 04

Data Visualization for Actuaries

A 90-Minute Practical Workshop

In this session, you will learn to visualize distributions, group comparisons and relationships with Pandas and Seaborn.

- **Part 1**: Plotting with Pandas
- **Part 2**: Plotting with Seaborn
- **Part 3**: Correlations and Pair Plots

The explanations use small tables created directly in the code. All exercises use `vehicle_fraud.csv` from Notebook 03.

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Seaborn style for all plots
sns.set_theme(style='whitegrid')

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

Part 1: Plotting with Pandas

The table `claims` with twelve claims is used for all explanations in Parts 1 to 3.

```
# Create a small DataFrame of claims
claims = pd.DataFrame({
    'age': [45, 38, 52, 61, 26, 47, 29, 68, 55, 41, 34, 58],
    'vehicle_type': ['Compact', 'SUV', 'Compact', 'Luxury', 'SUV', 'Luxury', 'Compact',
        'Compact', 'SUV', 'Luxury', 'Compact', 'SUV'],
    'vehicle_value': [14500, 38000, 12000, 85000, 35000, 64000, 13000, 11000, 32000, 72000,
        16500, 41000],
    'claim_amount': [1200.0, 4100.0, 800.0, 15000.0, 10500.0, 7200.0, 5400.0, 650.0, 2300.0,
        11500.0, 1500.0, 2800.0],
    'fraud': [0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0]
})

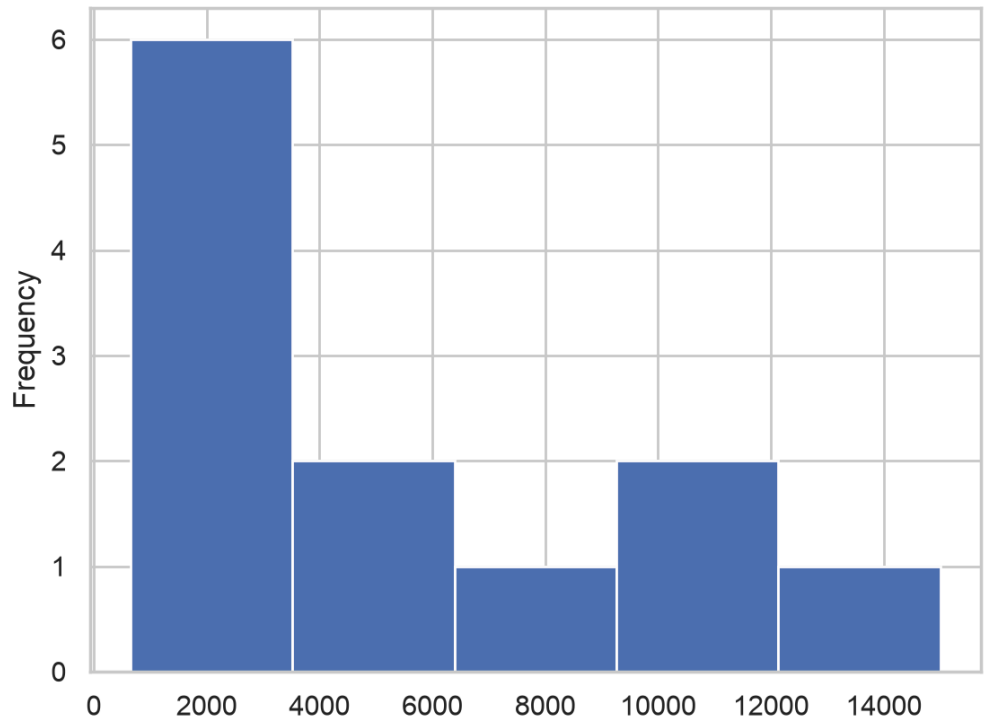
print(f"\n{sep}\nSmall DataFrame: claims\n{sep}")
print(claims)
```

```
================================================
Small DataFrame: claims
================================================
    age vehicle_type  vehicle_value  claim_amount  fraud
0    45      Compact          14500        1200.0      0
1    38          SUV          38000        4100.0      0
2    52      Compact          12000         800.0      0
3    61       Luxury          85000       15000.0      0
4    26          SUV          35000       10500.0      1
5    47       Luxury          64000        7200.0      0
6    29      Compact          13000        5400.0      1
7    68      Compact          11000         650.0      0
8    55          SUV          32000        2300.0      0
9    41       Luxury          72000       11500.0      0
10   34      Compact          16500        1500.0      0
11   58          SUV          41000        2800.0      0
```

1.1 Histograms

`.plot(kind=...)` draws a Series as a chart. A histogram (`kind='hist'`) divides the values into `bins` intervals and counts the values per interval. `plt.show()` displays the plot.

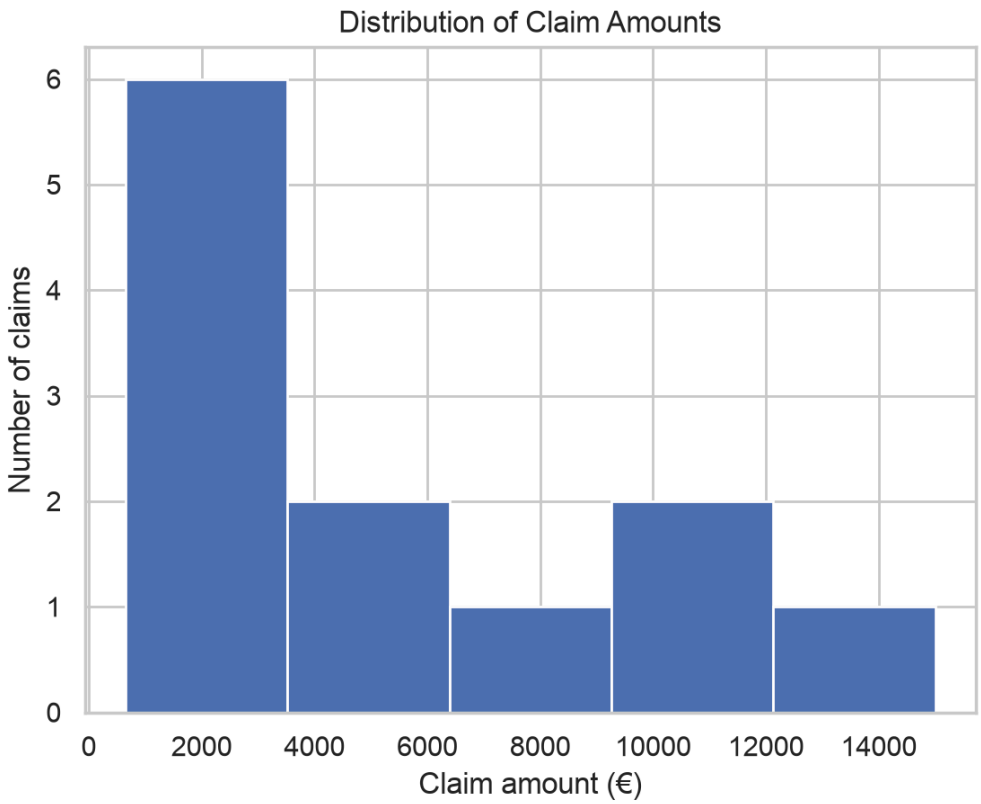
```
# Histogram of the claim amounts with 5 bins
claims['claim_amount'].plot(kind='hist', bins=5)
plt.show()
```



1.2 Titles and Axis Labels

`plt.title()`, `plt.xlabel()` and `plt.ylabel()` label the plot. Call them between the plot and `plt.show()`.

```
# Histogram with title and axis labels
claims['claim_amount'].plot(kind='hist', bins=5)
plt.title('Distribution of Claim Amounts')
plt.xlabel('Claim amount (€)')
plt.ylabel('Number of claims')
plt.show()
```



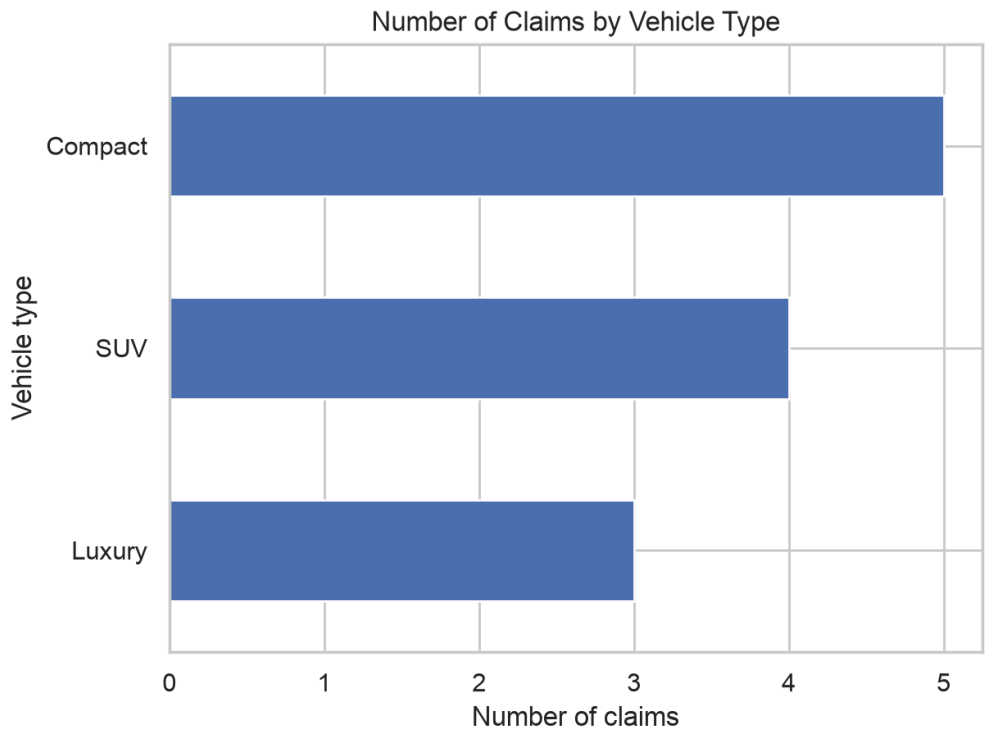
Most claims are small, a few are large: the distribution is **right-skewed**, which is typical for claim amounts.

1.3 Bar Charts

`kind='barh'` draws one horizontal bar per index label. Compute the values first with `value_counts()` or `groupby()`. `.sort_values()` puts the largest bar on top.

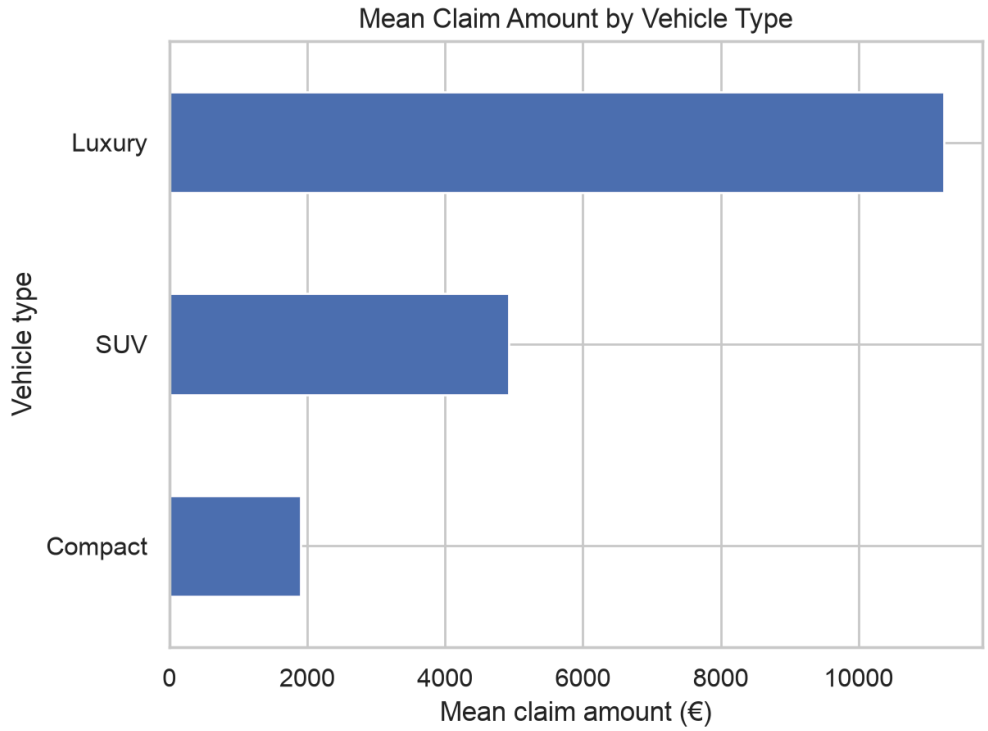
```
# Number of claims per vehicle type
counts = claims['vehicle_type'].value_counts().sort_values()

counts.plot(kind='barh')
plt.title('Number of Claims by Vehicle Type')
plt.xlabel('Number of claims')
plt.ylabel('Vehicle type')
plt.show()
```



```
# Mean claim amount per vehicle type
mean_claims = claims.groupby('vehicle_type')['claim_amount'].mean().sort_values()

mean_claims.plot(kind='barh')
plt.title('Mean Claim Amount by Vehicle Type')
plt.xlabel('Mean claim amount (€)')
plt.ylabel('Vehicle type')
plt.show()
```



1.4 Line Plots

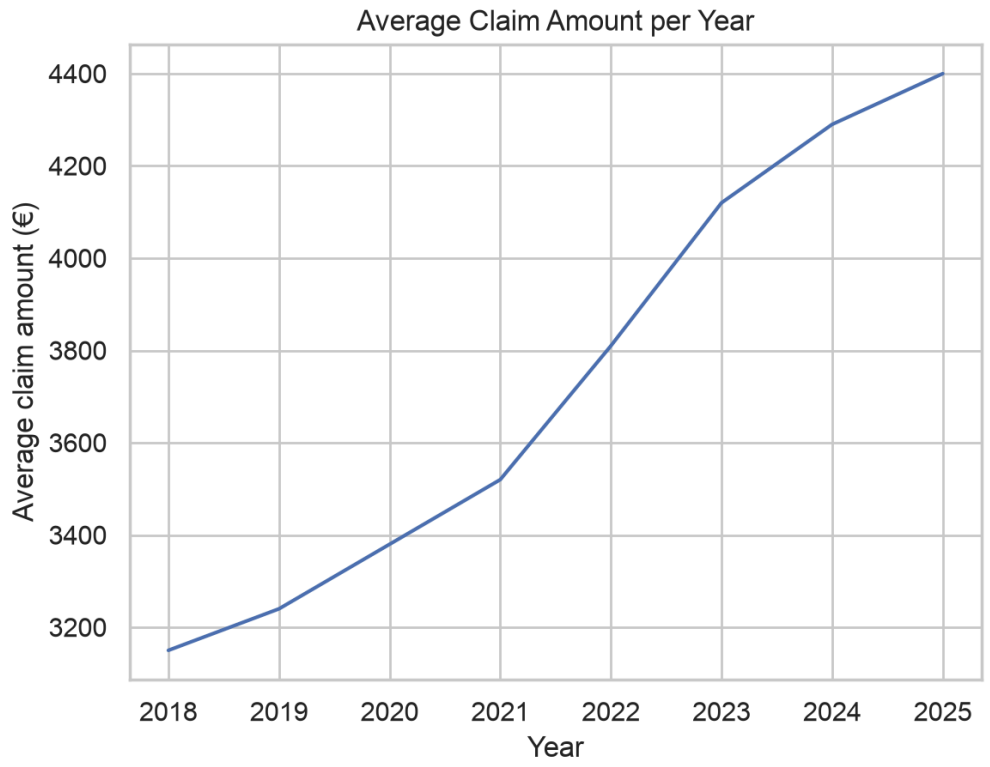
`kind='line'` connects the values in the order of the index, e.g. years.

```
# Average claim amount per year
average_claim = pd.Series([3150, 3240, 3380, 3520, 3810, 4120, 4290, 4400],
                          index=[2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025])

print(f"\n{sep}\nAverage Claim Amount per Year: average_claim\n{sep}")
print(average_claim)
```

```
==================================================
Average Claim Amount per Year: average_claim
==================================================
2018    3150
2019    3240
2020    3380
2021    3520
2022    3810
2023    4120
2024    4290
2025    4400
dtype: int64
```

```python
# Line plot of the average claim amount
average_claim.plot(kind='line')
plt.title('Average Claim Amount per Year')
plt.xlabel('Year')
plt.ylabel('Average claim amount (€)')
plt.show()
```



The average claim amount rose by 40% from 2018 to 2025: claims inflation.

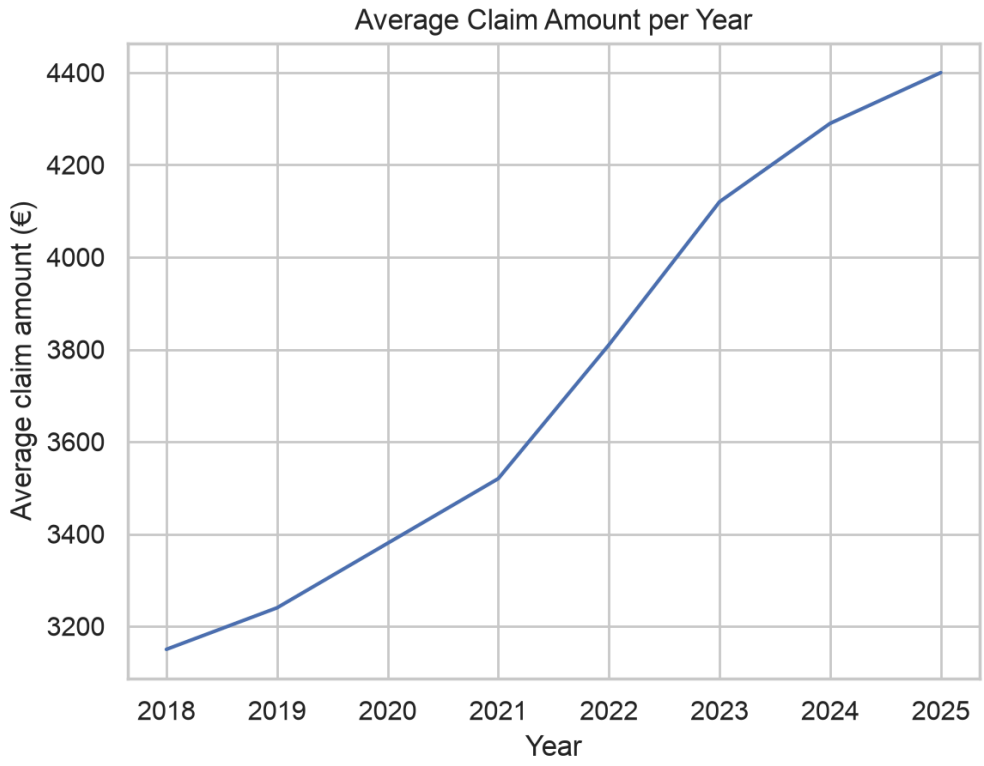
1.5 Saving a Plot

`plt.savefig('file.png')` saves the plot as an image file, e.g. for a report.

> **Watch out:** Call `plt.savefig()` before `plt.show()`. Afterwards, it saves an empty image.

```
# Save the line plot as a PNG file
average_claim.plot(kind='line')
plt.title('Average Claim Amount per Year')
plt.xlabel('Year')
plt.ylabel('Average claim amount (€)')
plt.savefig('average_claim.png')
print('Plot saved to average_claim.png')
plt.show()
```

```
Plot saved to average_claim.png
```



Exercise 1: Plotting `vehicle_fraud.csv` with Pandas

Show distributions and fraud rates with histograms, bar charts and a line plot, and save one plot as a file.

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
```

Task 1: Draw a histogram of `claim_amount` with 50 bins, a title and axis labels. Describe the shape.

```
# Task 1: Histogram of claim_amount
# Hint: df['claim_amount'].plot(kind='hist', bins=50)
#       plt.title('...'), plt.xlabel('...'), plt.ylabel('...')
#       plt.show()
```

Task 2: Draw a histogram of `np.log(df['claim_amount'])`. What changes compared to Task 1?

```
# Task 2: Histogram of the logarithm
# Hint: np.log(df['claim_amount']).plot(kind='hist', bins=50)
```

Task 3: Draw a bar chart of the number of claims per `vehicle_type`.

```
# Task 3: Number of claims per vehicle type
# Hint: counts = df['vehicle_type'].value_counts().sort_values()
#       counts.plot(kind='barh')
```

Task 4: Draw a bar chart of the fraud rate per `occupation`. Which occupation has the lowest rate?

```
# Task 4: Fraud rate per occupation
# Hint: fraud_rate = df.groupby('occupation')['fraud'].mean().sort_values()
#       fraud_rate.plot(kind='barh')
```

Task 5: Draw a line plot of the fraud rate per `number_of_prior_claims`.

```
# Task 5: Fraud rate per number of prior claims
# Hint: df.groupby('number_of_prior_claims')['fraud'].mean().plot(kind='line')
```

Task 6: Save the plot of Task 5 as `fraud_by_prior_claims.png`.

```
# Task 6: Save the plot of Task 5
# Hint: Repeat the plot and call plt.savefig('fraud_by_prior_claims.png') before plt.show()
```

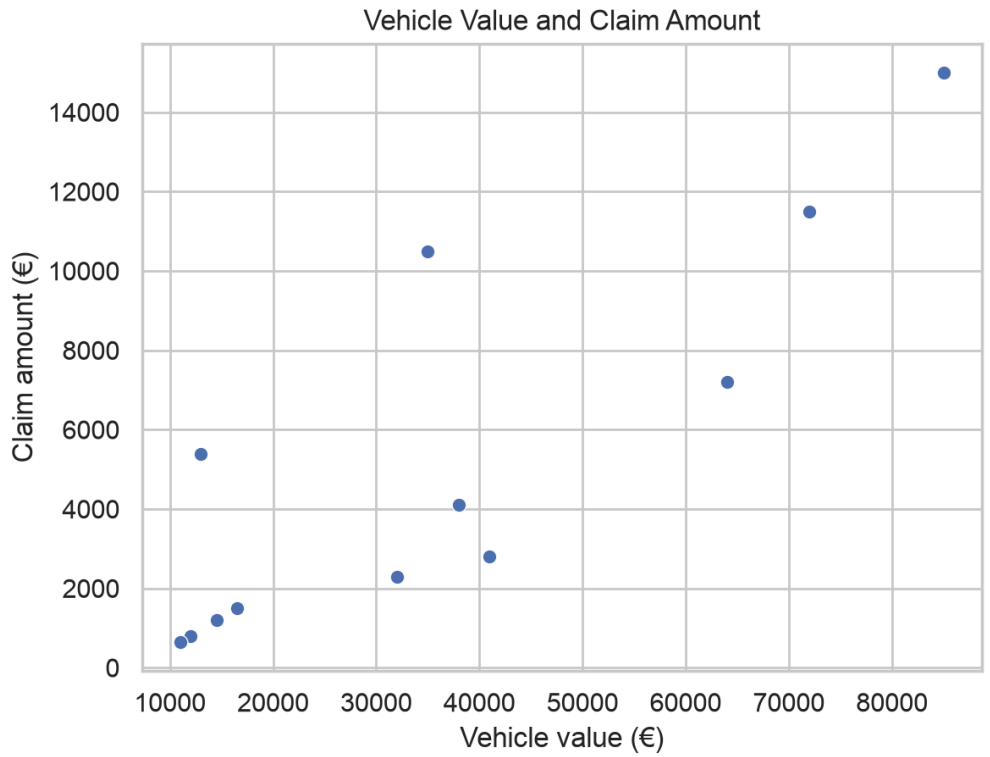
Part 2: Plotting with Seaborn

Seaborn draws plots directly from a DataFrame: `data=` names the table, `x=` and `y=` name the columns. Titles and axis labels work as in Part 1.

2.1 Scatter Plots

A scatter plot shows each row as a point. With many points, `alpha=0.2` makes the points transparent so that dense areas become visible.

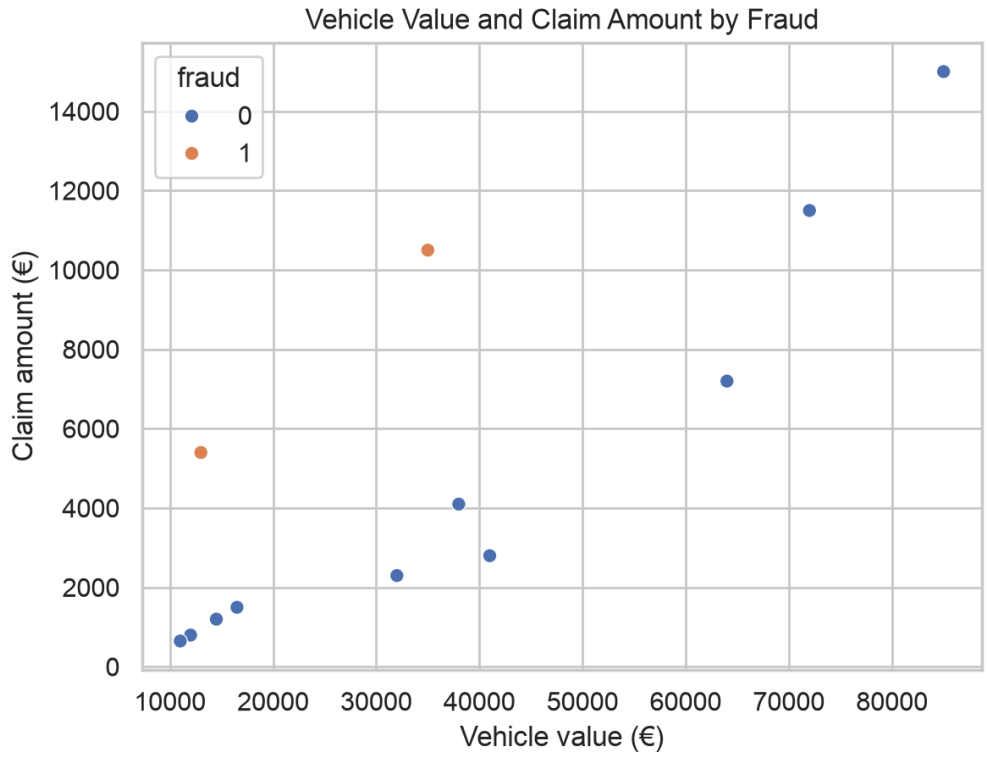
```
# Scatter plot: vehicle value and claim amount
sns.scatterplot(data=claims, x='vehicle_value', y='claim_amount')
plt.title('Vehicle Value and Claim Amount')
plt.xlabel('Vehicle value (€)')
plt.ylabel('Claim amount (€)')
plt.show()
```



2.2 Colors by Group

`hue=` colors the points by a column. It works in every Seaborn plot.

```
# Scatter plot colored by fraud
sns.scatterplot(data=claims, x='vehicle_value', y='claim_amount', hue='fraud')
plt.title('Vehicle Value and Claim Amount by Fraud')
plt.xlabel('Vehicle value (€)')
plt.ylabel('Claim amount (€)')
plt.show()
```



The two fraudulent claims are much higher than the other claims with similar vehicle values.

2.3 Box Plots

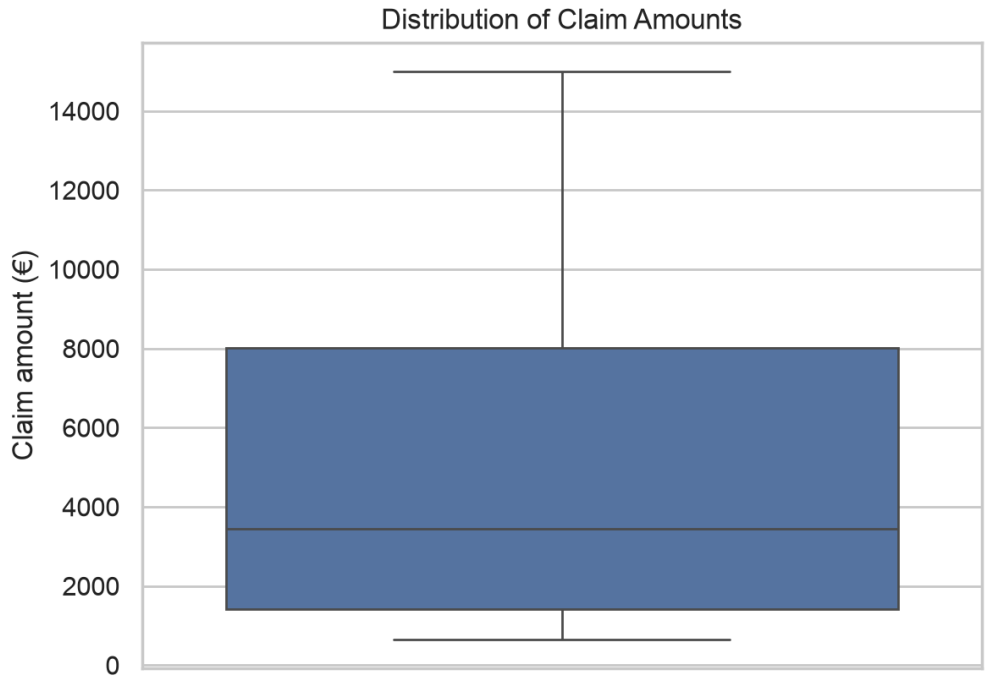
A box plot summarizes a distribution:

- The box spans the middle 50% of the values, from `25%` to `75%` in `describe()`.
- The line in the box is the median (`50%`).
- The whiskers reach the most extreme values within 1.5 box lengths of the box.
- Points beyond the whiskers are outliers.

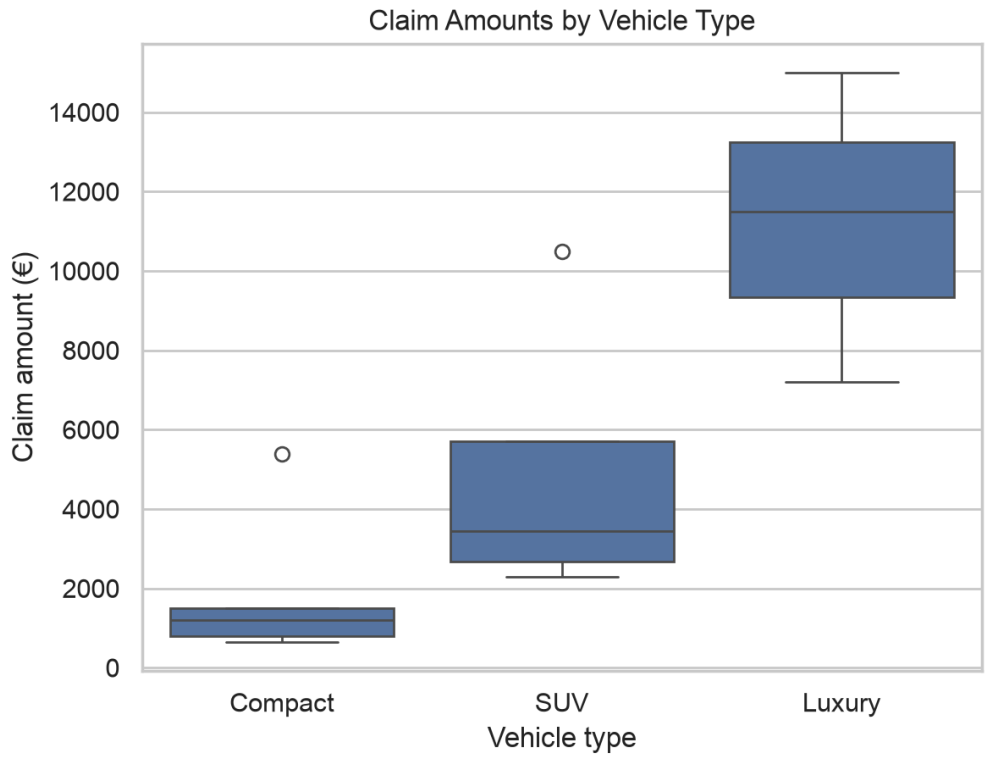
```python
# Quartiles of the claim amounts
print(f"\n{sep}\nStatistical Summary: claims['claim_amount'].describe()\n{sep}")
print(claims['claim_amount'].describe())
```

```
==================================================
Statistical Summary: claims['claim_amount'].describe()
==================================================
count       12.000000
mean      5245.833333
std       4795.521506
min        650.000000
25%       1425.000000
50%       3450.000000
75%       8025.000000
max      15000.000000
Name: claim_amount, dtype: float64
```

```python
# Box plot of the claim amounts
sns.boxplot(data=claims, y='claim_amount')
plt.title('Distribution of Claim Amounts')
plt.ylabel('Claim amount (€)')
plt.show()
```



```
# Box plot per vehicle type
sns.boxplot(data=claims, x='vehicle_type', y='claim_amount')
plt.title('Claim Amounts by Vehicle Type')
plt.xlabel('Vehicle type')
plt.ylabel('Claim amount (€)')
plt.show()
```



The outliers for Compact and SUV are the two fraudulent claims from Section 2.2.

Exercise 2: Plotting `vehicle_fraud.csv` with Seaborn

Compare fraudulent and legitimate claims with scatter plots and box plots in Seaborn.

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
```

Task 1: Draw a scatter plot of `schufa_score` (x) and `reporting_delay_days` (y) with `alpha=0.2`. Describe the relationship.

```
# Task 1: Scatter plot of schufa_score and reporting_delay_days
# Hint: sns.scatterplot(data=df, x='schufa_score', y='reporting_delay_days', alpha=0.2)
```

Task 2: Color the points of Task 1 by `fraud`. Where are the fraudulent claims?

```
# Task 2: Colored by fraud
# Hint: hue='fraud'
```

Task 3: Draw a box plot of `claim_amount` per `vehicle_type`. Does the vehicle type matter?

```
# Task 3: Claim amounts per vehicle type
# Hint: sns.boxplot(data=df, x='vehicle_type', y='claim_amount')
```

Task 4: Draw a box plot of `reporting_delay_days` per `police_report`, colored by `fraud`. What do you see?

```
# Task 4: Reporting delay per police report, colored by fraud
# Hint: sns.boxplot(data=df, x='police_report', y='reporting_delay_days', hue='fraud')
```

Part 3: Correlations and Pair Plots

3.1 Correlation Matrix

`corr()` measures the linear relationship between two columns on a scale from −1 to 1:

Value	Meaning
near 1	Both rise together
near −1	One rises, the other falls
near 0	No linear relationship

```
# Correlation matrix of three columns
corr = claims[['vehicle_value', 'claim_amount', 'fraud']].corr()

print(f"\n{sep}\nCorrelation Matrix: claims[['vehicle_value', 'claim_amount',
    'fraud']].corr()\n{sep}")
print(corr.round(2))
```

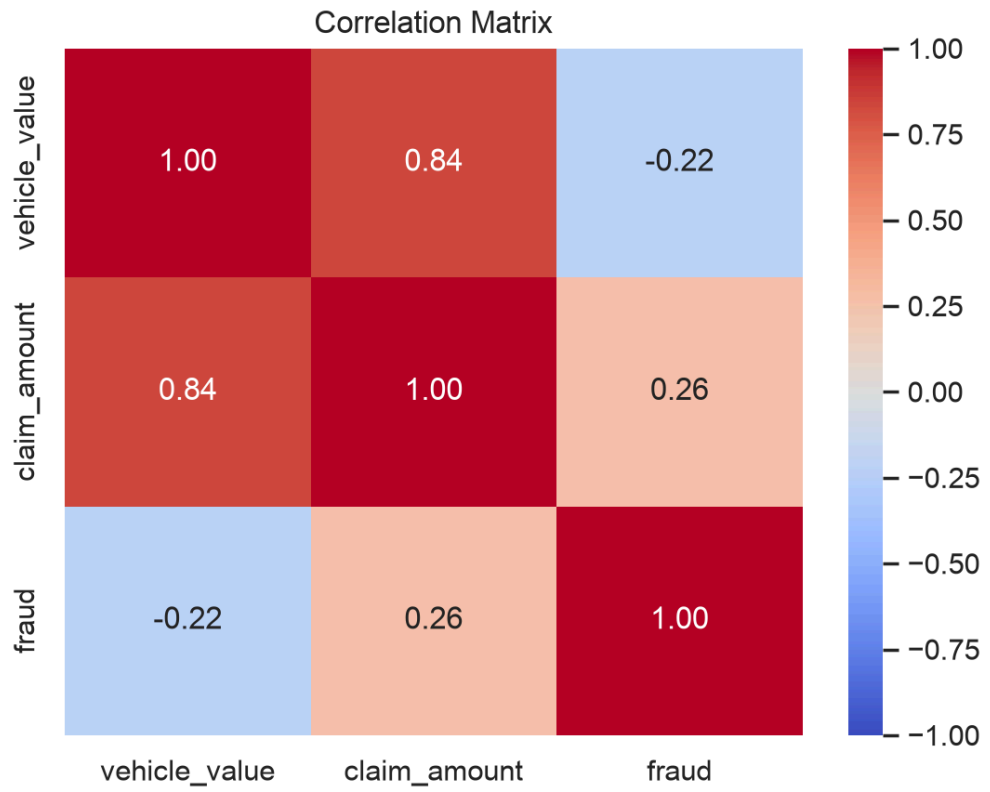
```
==================================================
Correlation Matrix: claims[['vehicle_value', 'claim_amount', 'fraud']].corr()
==================================================
               vehicle_value  claim_amount  fraud
vehicle_value           1.00          0.84  -0.22
claim_amount            0.84          1.00   0.26
fraud                  -0.22          0.26   1.00
```

3.2 Heatmaps

`sns.heatmap()` colors a matrix:

Parameter	Effect
`annot=True`	Writes the values into the cells
`fmt='.2f'`	Rounds them to two decimals
`cmap='coolwarm'`	Blue for negative, red for positive values
`vmin=-1, vmax=1`	Fixes the color scale from −1 to 1

```
# Heatmap of the correlation matrix
sns.heatmap(data=corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Matrix')
plt.show()
```

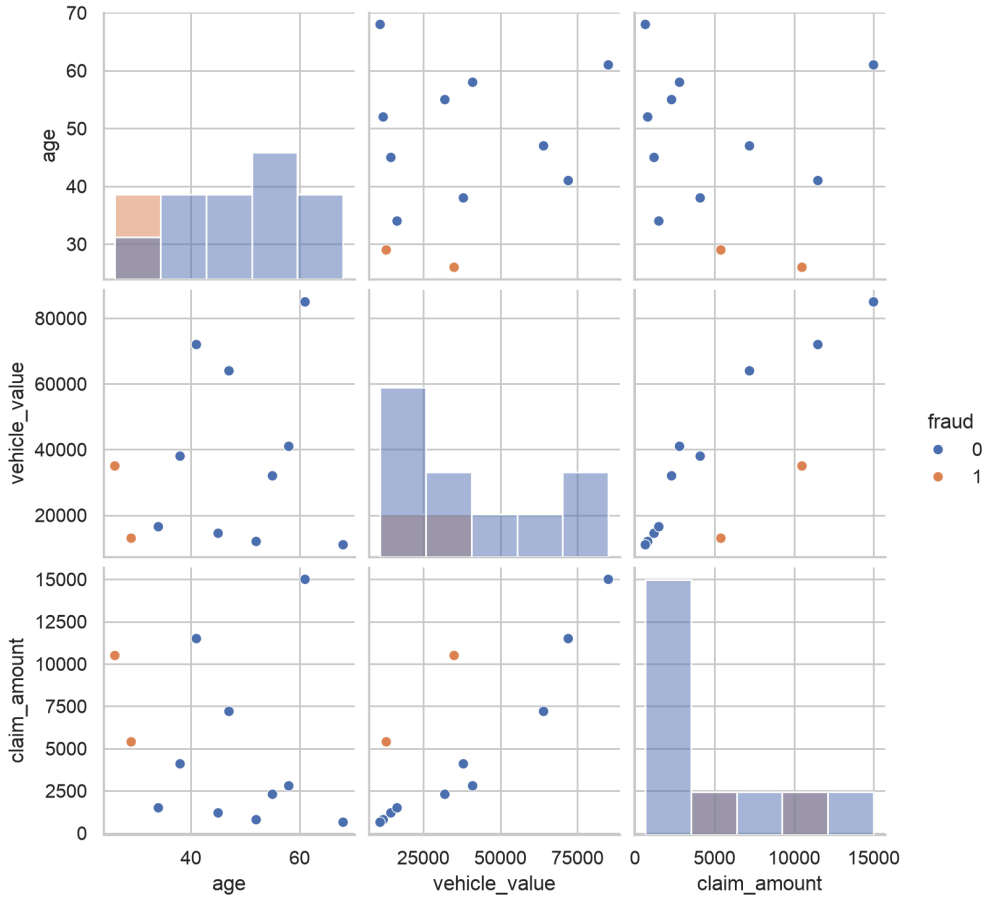


`vehicle_value` and `claim_amount` are strongly correlated (0.84), as the scatter plot in Section 2.1 showed.

3.3 Pair Plots

`sns.pairplot()` draws a scatter plot for every pair of numeric columns. `diag_kind='hist'` shows a histogram of each column on the diagonal.

```
# Pair plot of the numeric columns, colored by fraud
sns.pairplot(data=claims, hue='fraud', diag_kind='hist')
plt.show()
```



The two fraudulent claims come from the two youngest policyholders.

Exercise 3: Correlations in `vehicle_fraud.csv`

Measure the linear relationships between numeric columns and show them as a heatmap and a pair plot.

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
```

Task 1: Compute the correlation matrix of `policyholder_age`, `schufa_score`, `customer_since_years`, `number_of_prior_claims`, `reporting_delay_days` and `claim_amount`.

```
# Task 1: Correlation matrix
# Hint: cols = ['policyholder_age', 'schufa_score', ...]
#       corr = df[cols].corr()
```

Task 2: Draw it as a heatmap. Which two columns have the strongest correlation?

```
# Task 2: Heatmap
# Hint: sns.heatmap(data=corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
```

Task 3: Draw a pair plot of `schufa_score`, `reporting_delay_days` and `claim_amount`, colored by `fraud`.

```
# Task 3: Pair plot colored by fraud
# Hint: cols = ['schufa_score', 'reporting_delay_days', 'claim_amount', 'fraud']
#       sns.pairplot(data=df[cols], hue='fraud', diag_kind='hist')
```

Summary

- **Pandas:** `.plot(kind=...)` with `'hist'`, `'barh'` and `'line'`, labels with `plt.title()`, `plt.xlabel()`, `plt.ylabel()`, saving with `plt.savefig()`
- **Seaborn:** `sns.scatterplot()` and `sns.boxplot()` with `data=`, `x=`, `y=` and `hue=`
- **Correlations:** `corr()`, `sns.heatmap()`, `sns.pairplot()`

Next: preparing data for machine learning in Notebook 05.

- Pandas Visualization Guide
- Seaborn Tutorial